

Server-Factory-Core-Framework

内容

每个 **Server Factory** 背后共享的核心引擎。

概要

核心框架 (Core Framework) 是支撑 Server Factory 系列部署工具的 Kotlin 框架。它提供了通用的引擎和抽象层，供 Mail Server Factory 等项目构建，确保每个“工厂”复用同一套久经验证的基础，而非重复实现部署基础功能。

简要说明

Server Factory 生态系统底层的共享 Kotlin 框架。它为下游工厂（如 Mail Server Factory、Web Service Factory、SonarQube Factory 等）提供通用的部署引擎、连接抽象层及安装步骤机制。

详细说明

核心框架是让整个 Server-Factory 家族成为可能的隐形工程基石：每个独立“工厂”产品（Mail Server Factory、Web Service Factory、SonarQube Factory、Caching Proxy Factory）均构建于其上。Server Factory 采用声明式设计——用户通过配置描述所需基础设施，工厂则解析该描述，在目标系统上安装并初始化软件——而核心框架正是实现这一模式的共享机制所在：它包含连接和传输抽象层（可覆盖各类目标系统）、编码“软件如何部署”的安装步骤模型，以及所有工厂原本需自行实现的共享底层逻辑。它回答了多产品工具链最终都会面临的结构性问题——“共享引擎该置于何处？”——而一次性解决这一问题，正是维系家族统一性、避免分化为四种略有差异的部署工具的关键。通过将这些功能集中于一个 Kotlin 框架，家族避免了在不同产品中重复实现部署逻辑，并确保行为一致：核心框架中改进的连接类型或安装原语，将惠及所有下游工厂。其代码几乎完全由 Kotlin 构成（约 990KB 的 Kotlin 代码，外加一层薄薄的 Shell 层），体现了其作为代码库而非脚本集合的定位。下游仓库均将其作为标准依赖项引用

(Parallels-Utils、Qemu-Utils、Utils 及 Definitions 包均指向核心框架

仓库作为生态系统的核心）。其 README 文档有意保持精简——它是其他项目的基础设施，通过 `version.txt/version_code.txt` 进行版本管理——且早于后续的 AI 工作，成为组织成熟 DevOps 工具链传承的一部分。

为何构建它

每个部署工具都需要相同的核心功能：连接目标系统的方式，以及安装/配置软件的步骤。若每个产品单独实现，将导致行为分化、缺陷倍增。核心框架将其集中化，让所有工厂共享同一个可靠的引擎。

为何它是变革性的

它是整个家族中杠杆效应最高的环节：在此处强化的连接类型或改进的安装原语，能将其正确性和功能即时传递给所有工厂，使整个工具链从单一投资中持续受益。这是“一次构建，全面复用”理念在最具回报的领域——基础设施自动化的基础层——的应用，在此处修正一个问题，即可修复下游的一切。

内容

创新之处

- 统一的可复用部署框架，抽象化连接与安装步骤逻辑。
- 清晰分离引擎（核心框架）与产品专用工厂。
- 版本固定分发（`version.txt/version_code.txt`），确保可复现的消费流程。

挑战与解决方案

- 避免部署逻辑重复：通过将共享机制提取至统一框架，供所有工厂调用解决。
- 跨产品行为一致性：通过通用抽象层确保连接类型与步骤在各处表现一致。
- （未验证）：特定内部 API 未在公开 README 中记录；除“共享框架由工厂调用”外，接口细节视为未验证。

技术栈（选择原因与实现方式）

- **Kotlin** —— 整体框架（约 990 KB）；属于 Server Factory 语系。
- **Shell** —— 辅助脚本，精简实用。
- **Gradle** —— 构建工具链（与该语系 `./gradlew` 使用方式一致）。

注：GitHub 标记该仓库为 Server-Factory 组织下的分支。非 AI 为中心，而是作为部署工具链的核心支撑。